

SESIÓN 01

# Deep Learning

## Introducción

Dr. León Felipe Palafox Novack

lfpalafox@up.edu.mx

01 Representaciones

02 Redes neuronales

03 Backpropagation

04 Redes convolucionales

05 Redes recurrentes

06 Transformers

02

# Anuncios

## AVISOS DE LA SESIÓN

Da clic en una línea para escribir. Se guarda en este navegador.

01

---

02

---

03

---

03

# Objetivos de la sesión

Al terminar la clase deberían poder:

01

## **Explicar**

cómo una red aprende representaciones.

02

## **Distinguir**

qué hace la función de pérdida y qué hace el optimizador.

03

## **Implementar y validar**

una red en PyTorch.

04

# Metodología de aprendizaje

01

## Teoría

---

Intuición

---

Mecanismo

---

Límites

---

02

## Práctica

---

Implementar

---

Evaluar

---

Interpretar

---

03

## Casos de uso

---

Fallas

---

Oportunidades

---

Cada clase tiene una parte de teoría y una de práctica. En la práctica revisamos casos reales: dónde funcionan los algoritmos y dónde fallan.

05

# Mapa del curso



06

# Evaluación

60%

Proyecto final

40%

Tareas y participación

Equipos de hasta 3 personas

Reporte de 3–5 páginas: datos, diseño y variables

Tareas

Participación en clase

07

# Proyecto final

01 **Escojan un problema concreto.**

02 **Entrenen un modelo para resolverlo.**

03 **Midan qué tan bien funciona.**

## IDEAS DE PROYECTO

### DEPORTES

#### Despeje óptimo del portero

¿A dónde patear para que su equipo gane el balón?

### VISIÓN

#### Detección de autos

Localizar vehículos en video de calle

### VISIÓN

#### Reconocimiento facial

Verificar identidad a partir de una foto

### VISIÓN

#### Generación de arte

Transferir el estilo de una pintura a una foto

### AUDIO

#### Generación de música

Componer secuencias nuevas

### TEXTO

#### Generación de texto

Escribir con el estilo de un autor

### TEXTO

#### Emojificador

Elegir el emoji adecuado para una frase

### TEXTO

#### Traducción automática

Pasar un texto de un idioma a otro

### AUDIO

#### Palabra de activación

Encender un sistema al oír una palabra

08

# Contenido de esta sesión

1

## Aplicaciones del aprendizaje profundo

AlphaFold y modelos de lenguaje

2

## PyTorch y otros frameworks

Tensores, diferenciación automática y GPU

3

## Fundamentos de redes neuronales

Perceptrón, activaciones y capas

4

## Backpropagation y optimización

Regla de la cadena y gradiente descendente

5

## CNN y RNN

Visión y secuencias

# Aprendizaje profundo

---

¿Cómo aprende una red la representación que necesita?

10

# El problema de clasificación

- 1 ¿A qué clase pertenece cada observación?
- 2 ¿Dónde se atora nuestro clasificador?
- 3 ¿Y si cambiamos cómo representamos los datos?

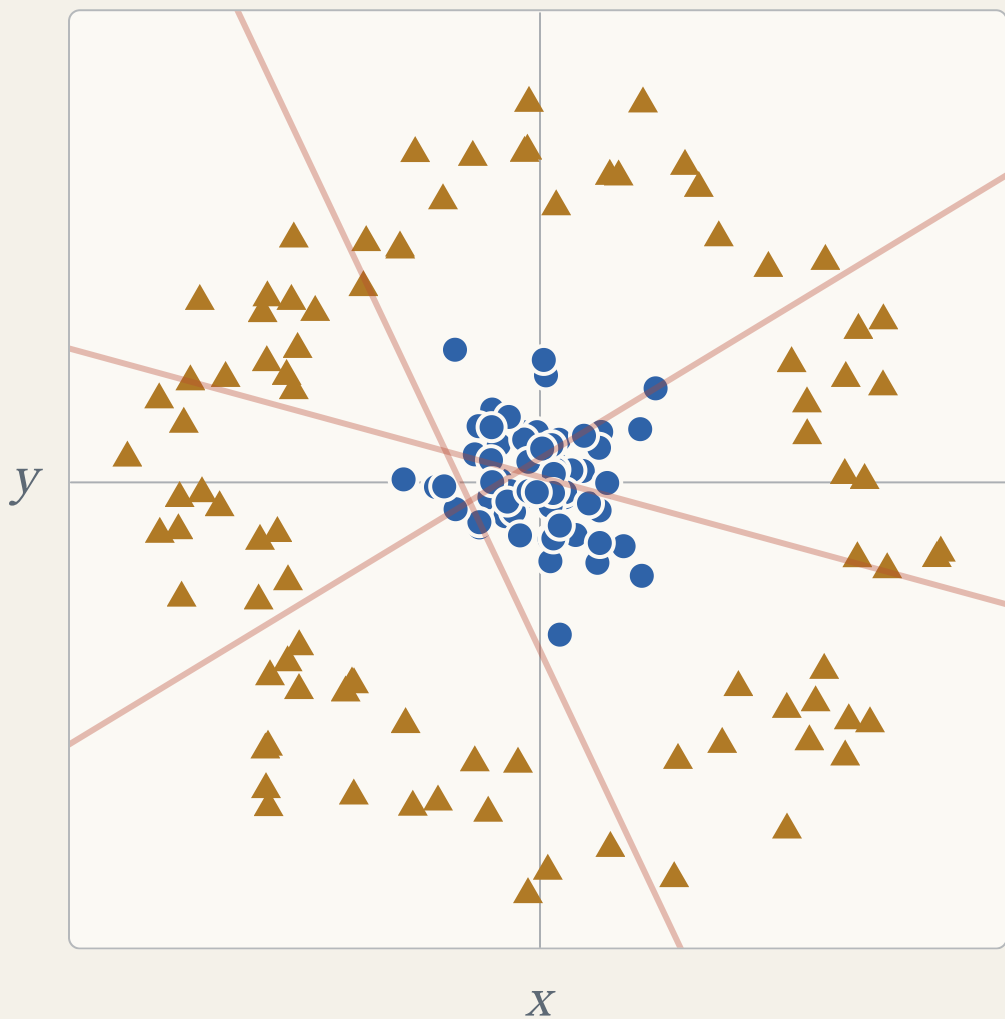
11

# La representación cambia el problema

Con las coordenadas originales,  
una línea recta no alcanza.

● Clase A    ▲ Clase B

✕ No hay recta que separe el centro  
del anillo.



12

# El mismo dato, otras coordenadas

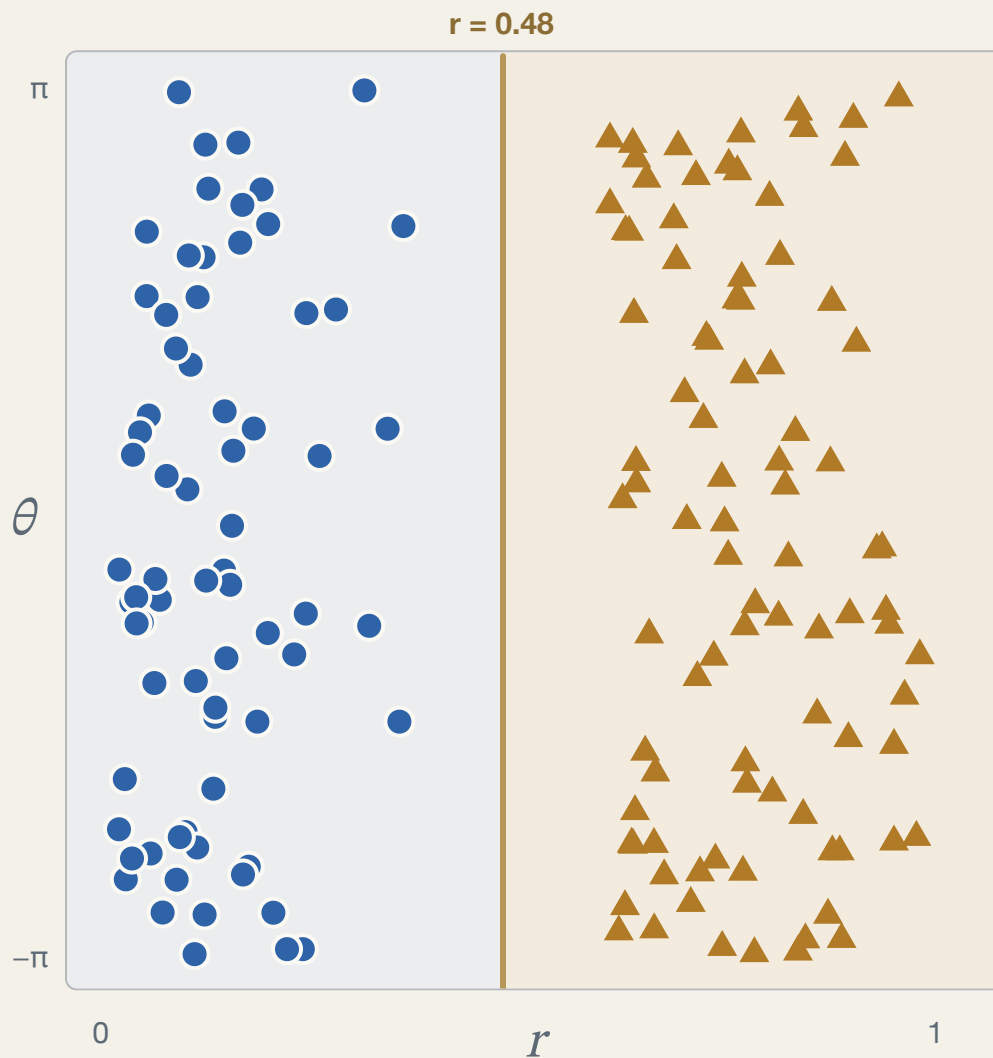
Si usamos el radio, los círculos concéntricos se separan solos.

● Clase A ▲ Clase B

$$r = \sqrt{x^2 + y^2}$$

$$\theta = \text{atan2}(y, x)$$

✓ Una recta vertical,  $r = 0.48$ , separa las clases.



# Clasificación y características

Las características (features) describen al dato: píxeles, palabras, medidas.

Con la representación original, a veces las clases no se pueden separar.

Si transformamos los datos, clasificar se vuelve más fácil.

## UNA OBSERVACIÓN → UN VECTOR DE CARACTERÍSTICAS

|               |                    |                                 |
|---------------|--------------------|---------------------------------|
| <b>Imagen</b> | píxeles            | $x = [0.12, 0.80, 0.33, \dots]$ |
| <b>Texto</b>  | conteo de palabras | $x = [0, 2, 1, 0, \dots]$       |
| <b>Sensor</b> | medidas            | $x = [36.8, 72, 1.2, \dots]$    |

## REPRESENTACIÓN INICIAL



## DESPUÉS DE UNA TRANSFORMACIÓN $\phi(x)$

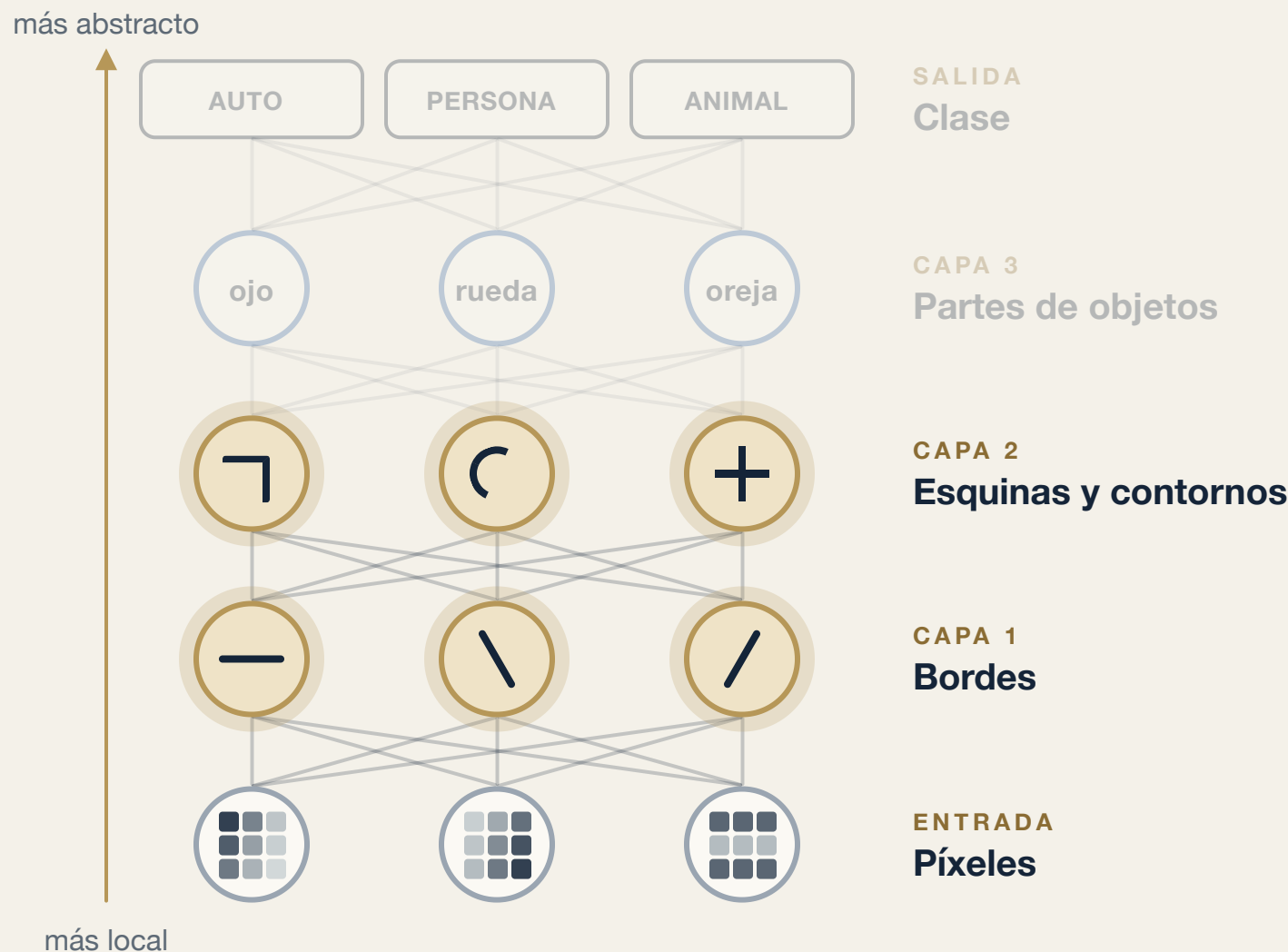


14

# Capas de representación

## Las primeras capas detectan patrones locales.

Encuentran cosas simples, como bordes, esquinas y contornos, en cualquier parte de la imagen.

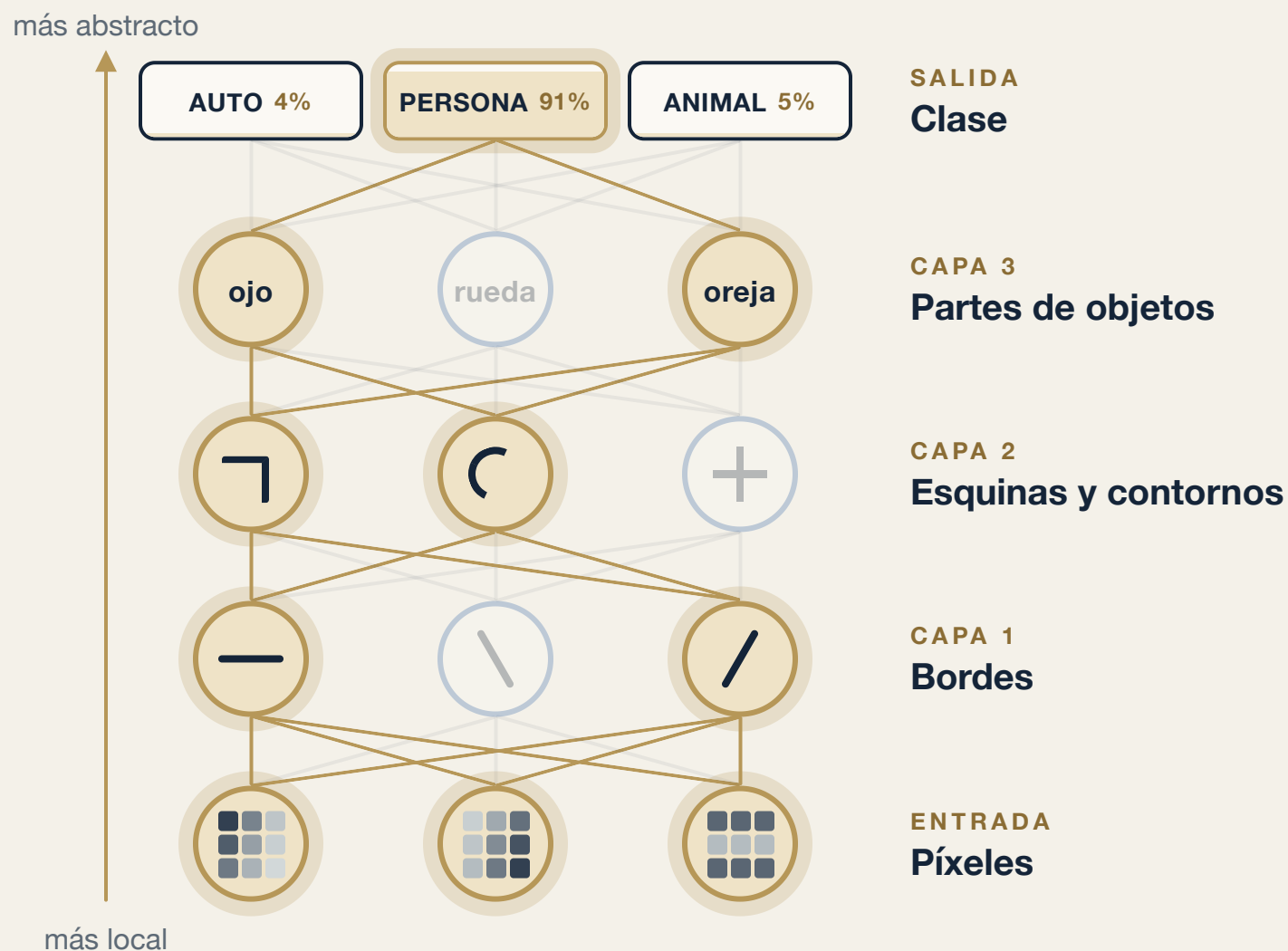


15

# La combinación de patrones

**Las capas siguientes juntan esos rasgos hasta reconocer la clase.**

Bordes → esquinas → partes (ojo, oreja) → **persona**.  
Cada capa trabaja con lo que encontró la anterior.



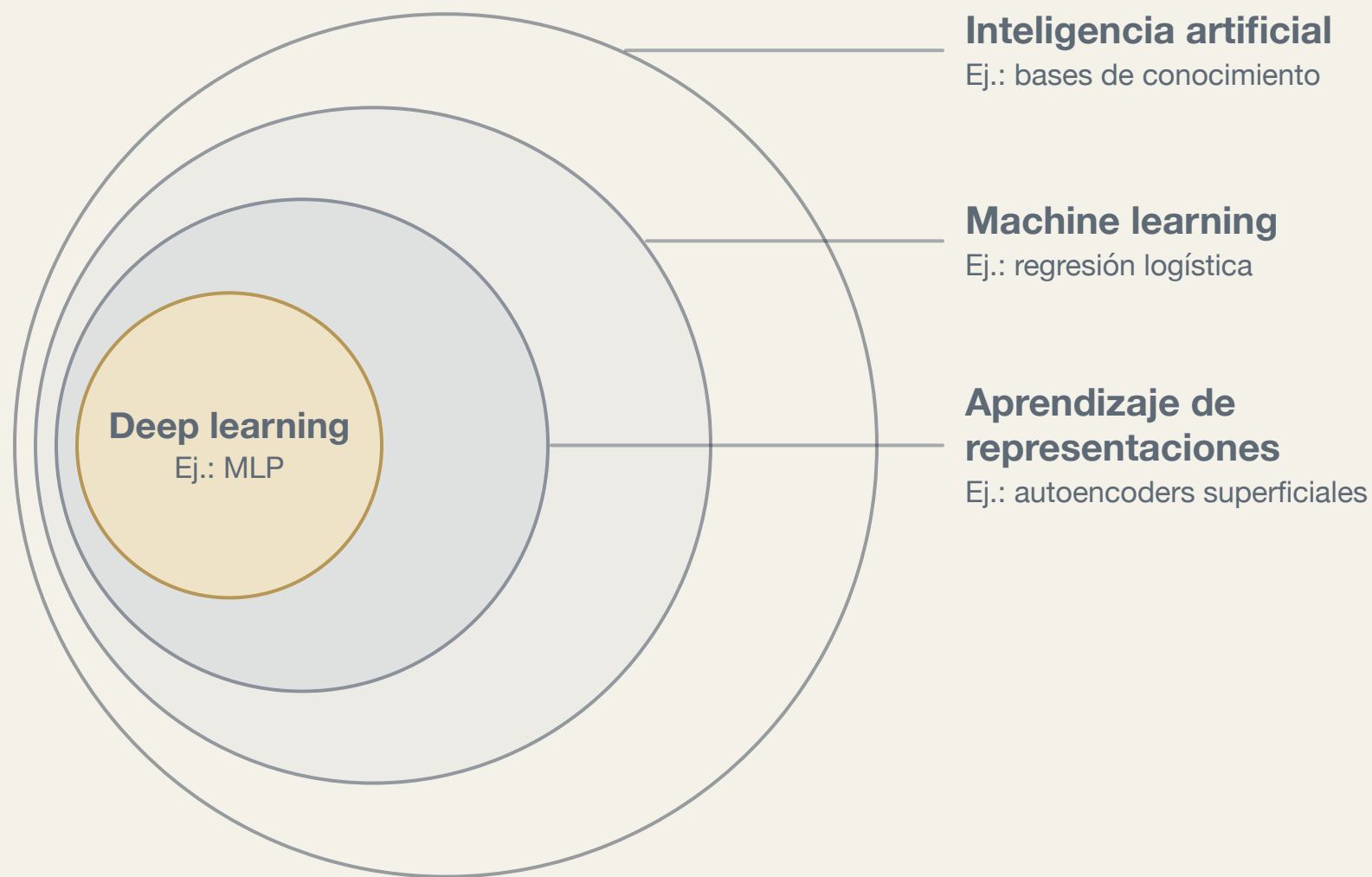
16

# IA, machine learning y deep learning

**Deep learning es un tipo de machine learning.**

$$DL \subset RL \subset ML \subset IA$$

De afuera hacia adentro: aprender de los datos, aprender también la representación, y hacerlo con muchas capas.



17

# Vocabulario de la clase

## Redes neuronales profundas

Modelos con varias capas.

deep learning

redes profundas

redes neuronales

## Neuronas o unidades

Cada una hace un cálculo con sus parámetros:

$$h = f(w^T x + b)$$

## Formas de aprendizaje

### Supervisado

pares (x, y) etiquetados

### Autosupervisado

la etiqueta sale del mismo dato

### Por refuerzo

recompensas de un entorno

18

# Prerrequisitos

1

Regresión logística

$$\hat{y} = \sigma(w^T x + b)$$

2

Gradiente descendente

$$w \leftarrow w - \eta \nabla L(w)$$

3

¿Qué función minimiza el optimizador?

$$L(w) = ?$$

19

# Ventajas y límites

## VENTAJAS

- + La red aprende las características; ya no hay que diseñarlas a mano.
- + Las GPU hacen muy rápido todo lo que se puede paralelizar.

## LO QUE SIGUE HACIENDO FALTA



Aunque la red aprenda la representación, igual hay que conseguir datos, pagar el cómputo y validar el modelo.

20

# Herramientas para implementarlo

Programamos en Python.

Corremos en nuestra máquina o en la nube.

LENGUAJE

`import torch`

FRAMEWORKS

PyTorch

TensorFlow

JAX

Keras

INFRAESTRUCTURA

LOCAL

laptop o estación de trabajo

NUBE

 Microsoft Azure  Google Colab

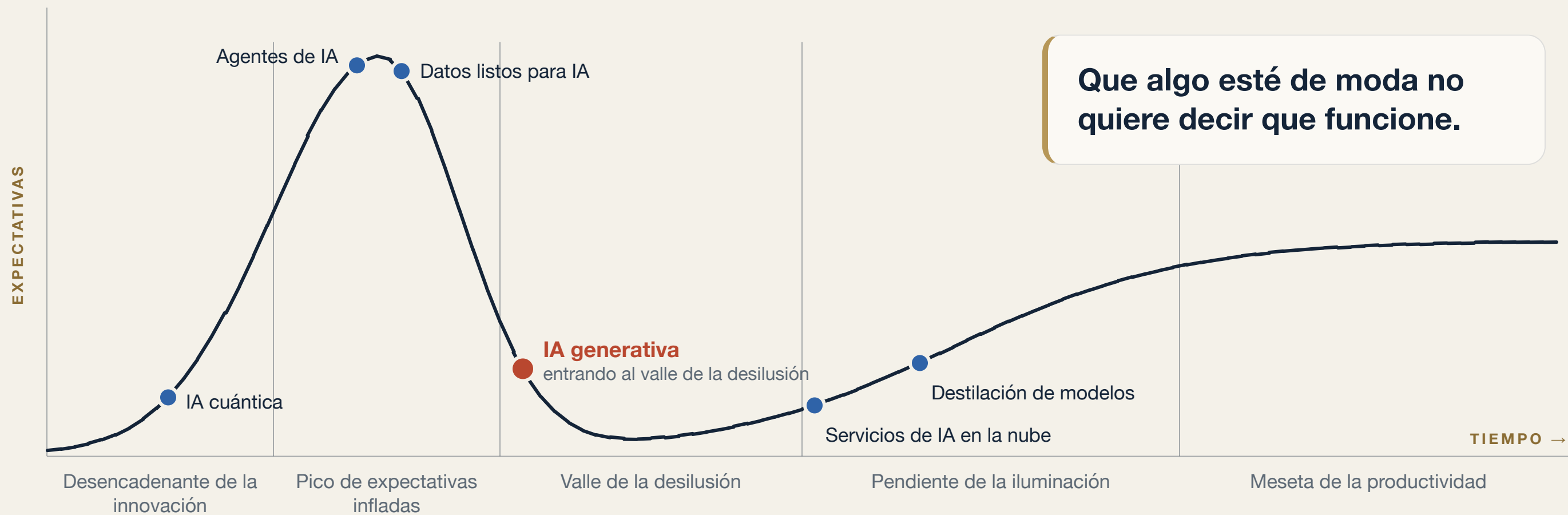
HARDWARE

CPU

GPU · CUDA

21

# Expectativas y madurez



Posiciones según Gartner, *Hype Cycle for Artificial Intelligence, 2025* (junio de 2025). Selección de tecnologías.

22

# GPU: el origen gráfico

Las GPU se hicieron para dibujar gráficos.

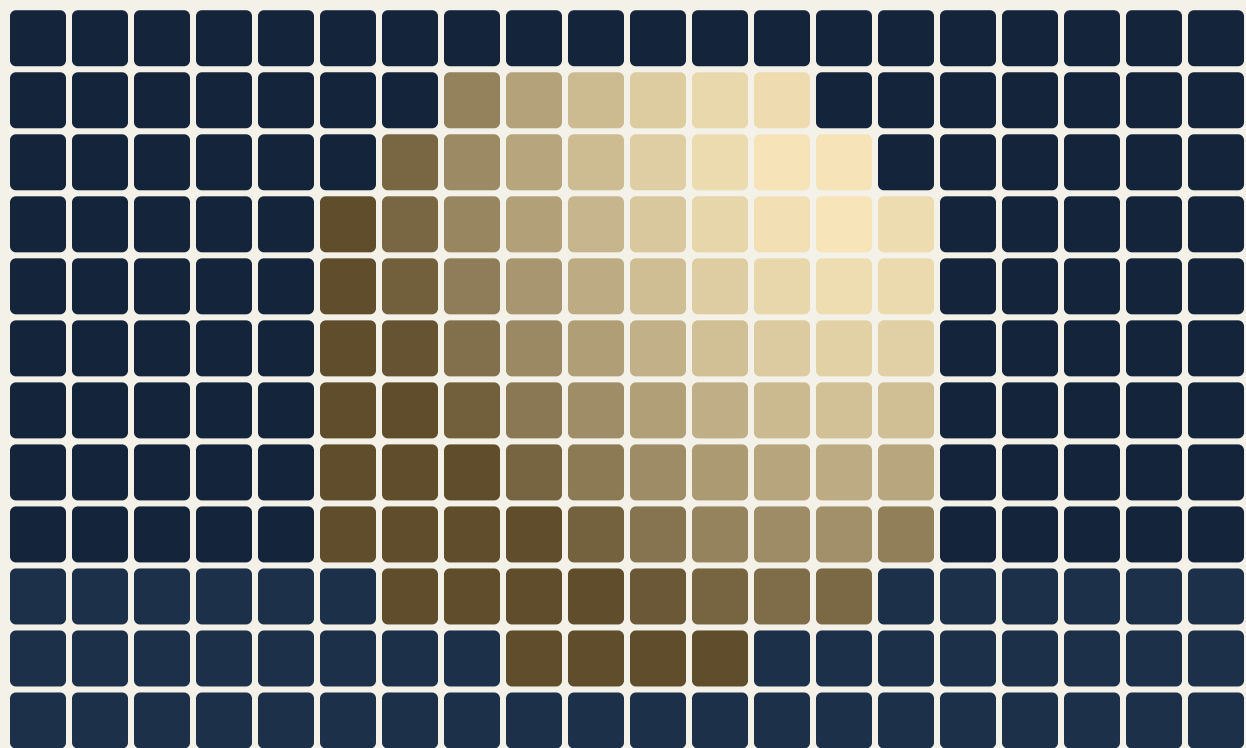
**1993**  **NVIDIA** se funda.

**1998** 3dfx Voodoo2 populariza la aceleración 3D en PC.

**1999** GeForce 256: NVIDIA la presenta como «la primera GPU».

*En ese entonces nadie las usaba para multiplicar matrices.*

LO QUE HACE LA GPU: CALCULAR CADA PÍXEL MUY RÁPIDO



fotograma 3

Cada píxel se calcula por separado, así que miles de cálculos iguales corren en paralelo.

23

# Cómputo paralelo y CUDA



**NVIDIA abrió sus tarjetas de video para hacer cálculo científico.**

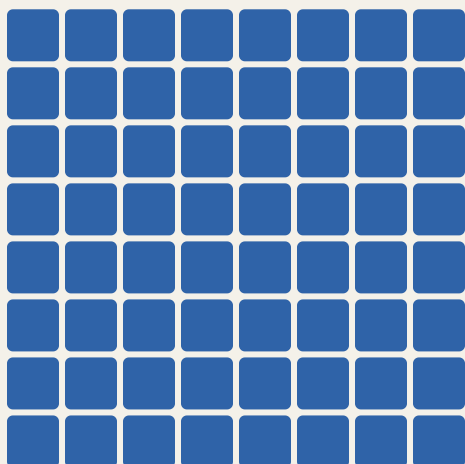
**2006** Sale la GeForce 8800 GTX, con 128 núcleos programables.

**2007** Sale CUDA y la GPU se puede programar para cualquier cálculo.

Detrás de CUDA estaban Ian Buck (venía de Stanford) y John Nickolls.

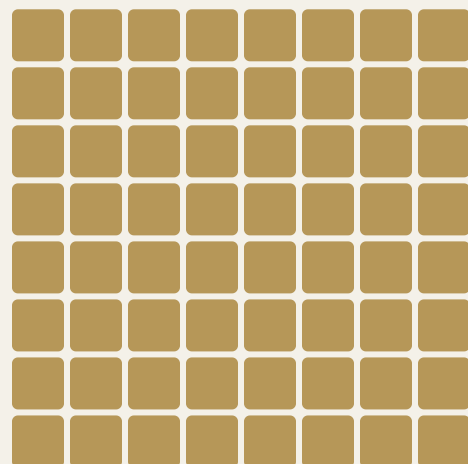
$C = A \times B$  · cada celda  $C_{ij} = \sum_k A_{ik} B_{kj}$  es independiente

**CPU · 4 núcleos**



pasos: **16** ✓

**GPU · 128 núcleos**



pasos: **1** ✓

La misma multiplicación: 16 pasos en CPU, 1 en GPU.

24

# Datos, cómputo y redes profundas

Lo hicieron tres personas  
con dos tarjetas de video.

## EQUIPO

Alex Krizhevsky, Ilya Sutskever y Geoffrey Hinton



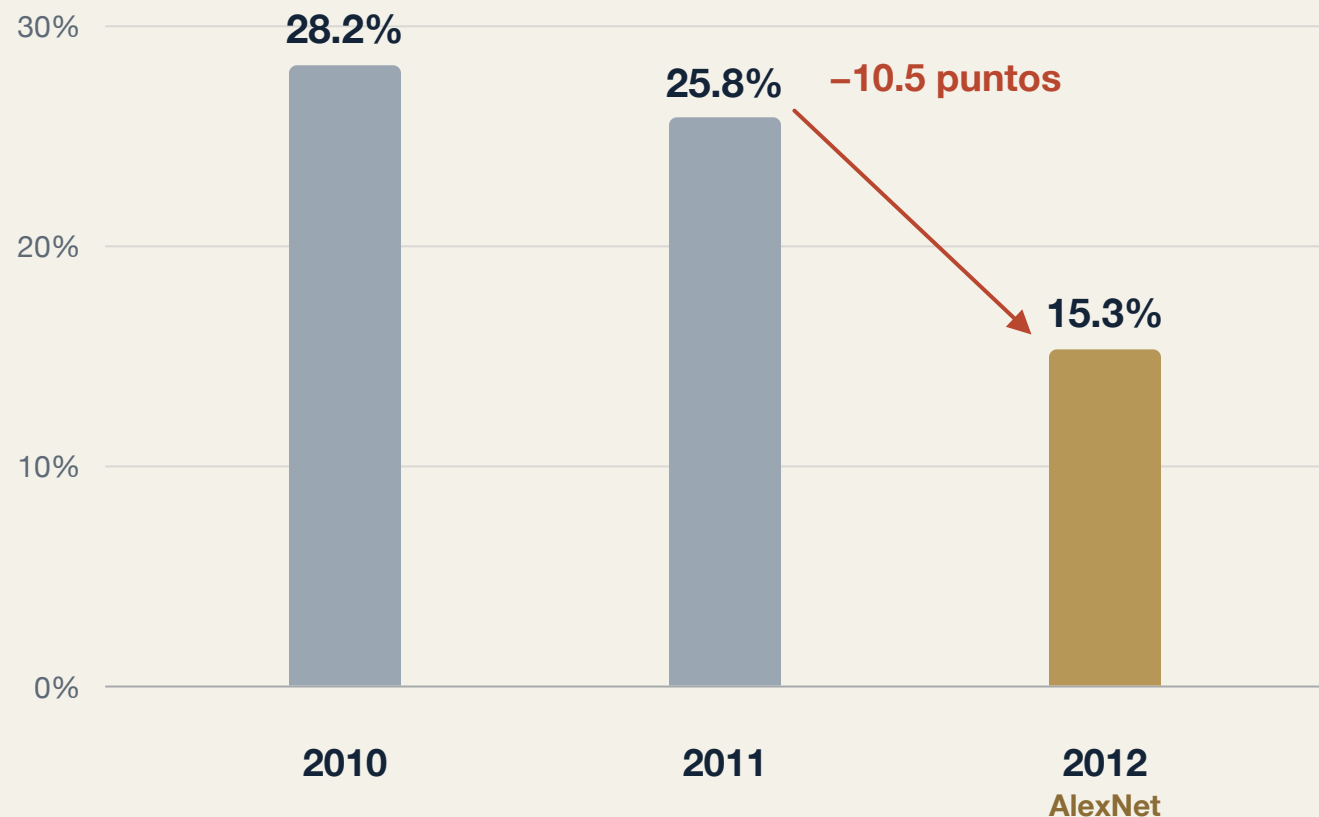
## CÓMPUTO

2 GPU NVIDIA GTX 580 (3 GB) · 5–6 días de  
entrenamiento

## ARTÍCULO

«ImageNet Classification with Deep Convolutional  
Neural Networks», NeurIPS 2012

ERROR TOP-5 DEL GANADOR DE ILSVRC (IMAGENET)



25

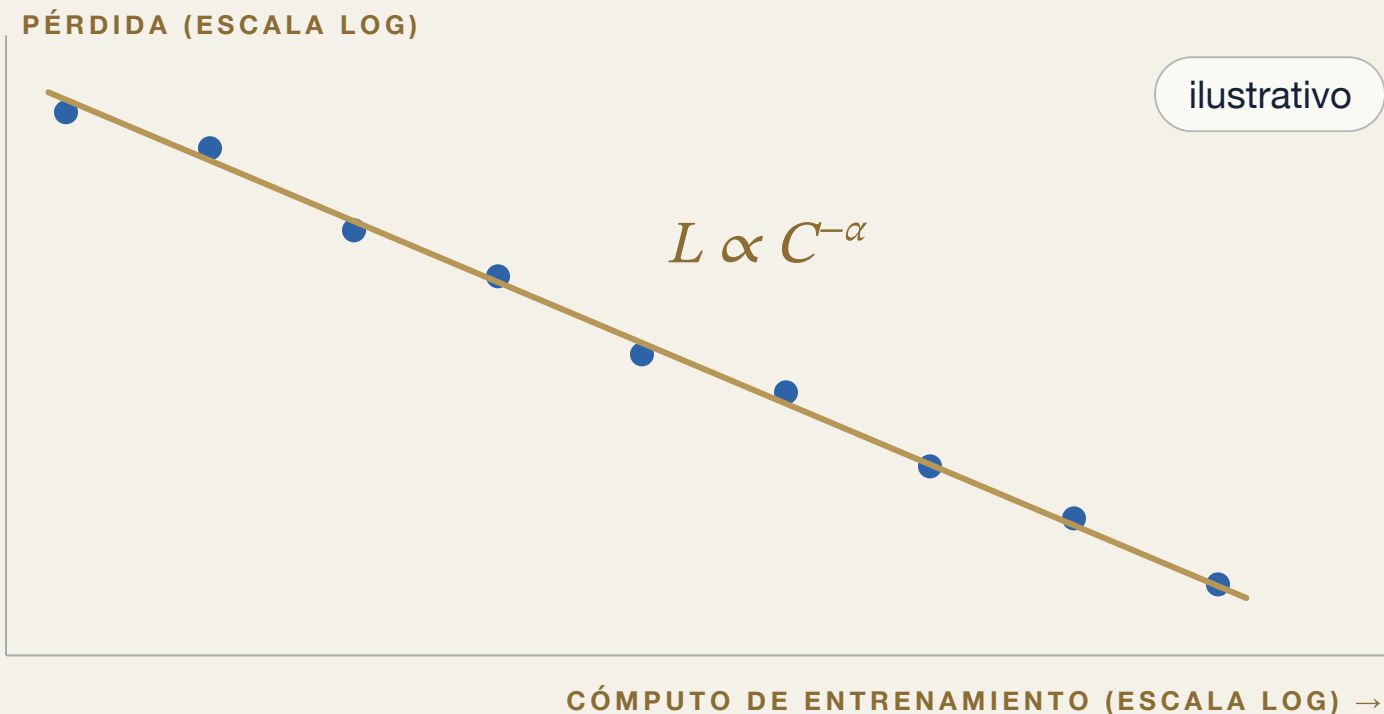
# La escala del entrenamiento

Con las leyes de escalamiento, lo que limita es cuántas GPU tienes.

KAPLAN ET AL., 2020

Al crecer los parámetros, los datos y el cómputo, la pérdida de un modelo de lenguaje baja siguiendo una ley de potencia.

Si más cómputo da mejores modelos de forma predecible, todos quieren más GPU.



V100 · 2017



A100 · 2020



H100 · 2022

26

# AlphaFold

Predice cómo se pliega una proteína.

AlphaFold quedó primero en CASP13 y AlphaFold 2 en CASP14.

AlphaFold 2 se basa en mecanismos de atención.

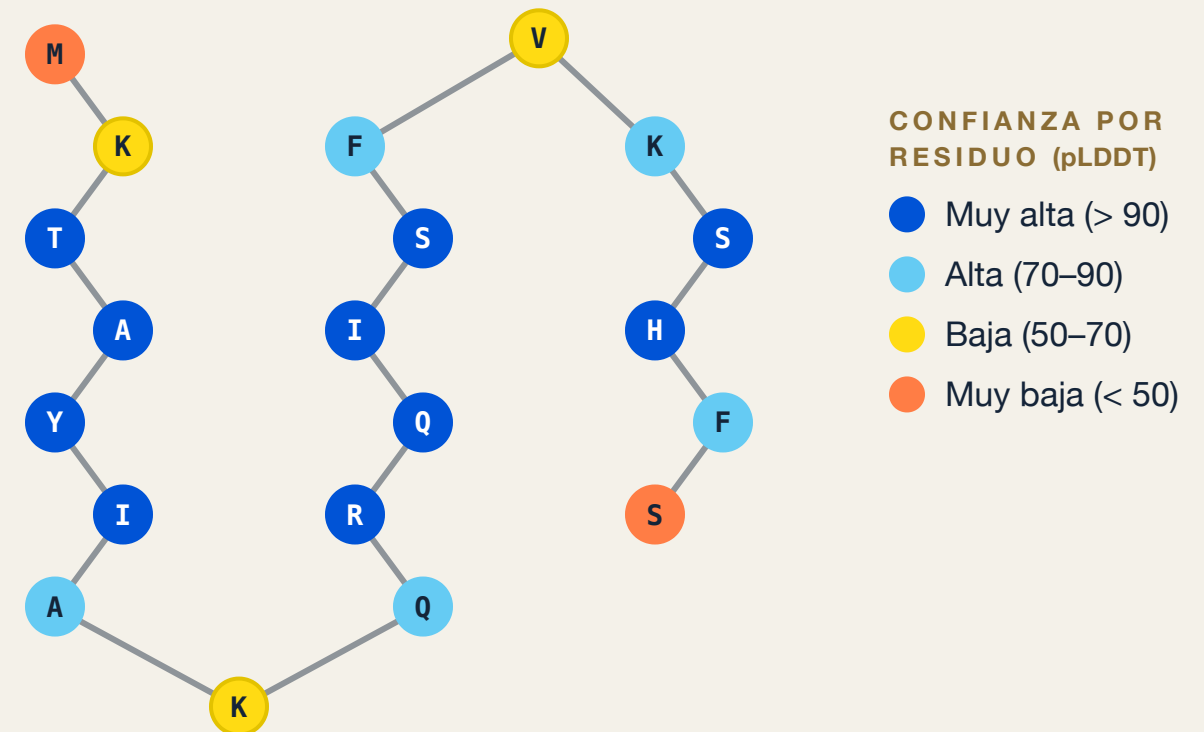


**CASP13 · 2018** AlphaFold

**CASP14 · 2020** AlphaFold 2

**2024** Nobel de Química: Hassabis y Jumper

ESTRUCTURA PREDICHA · TRES HÉLICES



27

# Modelos de lenguaje

Un Transformer aprende cómo se usa el lenguaje.

Genera texto palabra por palabra a partir del contexto.

Que suene bien no quiere decir que sea cierto.

## CONTEXTO

Las redes neuronales profundas aprenden representaciones jerárquicas y fueron inventadas en 2012.

## SIGUIENTE PALABRA · PROBABILIDAD (ILUSTRATIVA)

|             |                        |     |
|-------------|------------------------|-----|
| jerárquicas | <div><div></div></div> | 41% |
| útiles      | <div><div></div></div> | 30% |
| complejas   | <div><div></div></div> | 17% |
| a           | <div><div></div></div> | 7%  |



**Suena bien, pero es falso.** Las redes neuronales existen desde finales de los cincuenta; en 2012, con AlexNet, fue cuando despegaron.

28

# PyTorch

Librería de Python para construir y entrenar redes.

Trae tensores, derivadas automáticas y soporte para GPU.

También existen TensorFlow, JAX y Keras.

TensorFlow

JAX

Keras

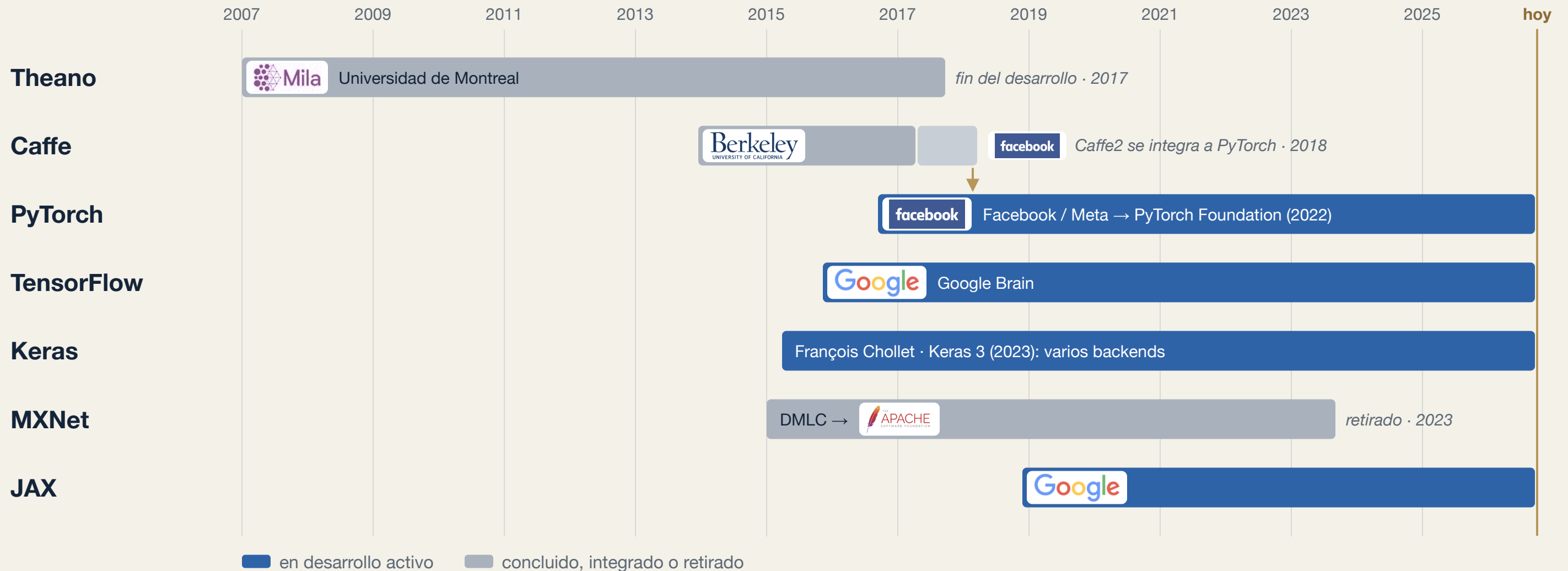
```
import torch
from torch import nn

# usa la GPU si está disponible
device = "cuda" if torch.cuda.is_available() else "cpu"
model = nn.Sequential(nn.Linear(3, 16), nn.ReLU(),
                      nn.Linear(16, 1)).to(device)
opt = torch.optim.SGD(model.parameters(), lr=0.1)

x = torch.randn(64, 3, device=device) # tensores
y = torch.randn(64, 1, device=device)
loss = nn.functional.mse_loss(model(x), y)
loss.backward() # diferenciación automática
opt.step()     # actualiza los pesos
```

29

# Historia de los frameworks



30

# El ecosistema de deep learning

## Theano



MILA, Universidad de Montreal (grupo de Yoshua Bengio)

De los primeros: grafos simbólicos y derivadas automáticas.

● Concluido · 2017

## Caffe → Caffe2



Yangqing Jia (UC Berkeley) → Facebook

Visión por computadora; Caffe2 para móviles y producción.

● Integrado a PyTorch · 2018

## PyTorch



Soumith Chintala y equipo (Facebook AI Research)

El grafo se arma mientras corre el código.

● Activo · PyTorch Foundation

## TensorFlow



Jeff Dean, Rajat Monga y Google Brain

Pensado para producción a gran escala.

● Activo

## MXNet



Tianqi Chen y la comunidad DMLC → Apache

Rápido y fácil de distribuir en varias máquinas.

● Retirado · 2023

## Hoy

Opciones principales

PyTorch, TensorFlow y JAX; Keras 3 funciona sobre los tres.

● Este curso: PyTorch

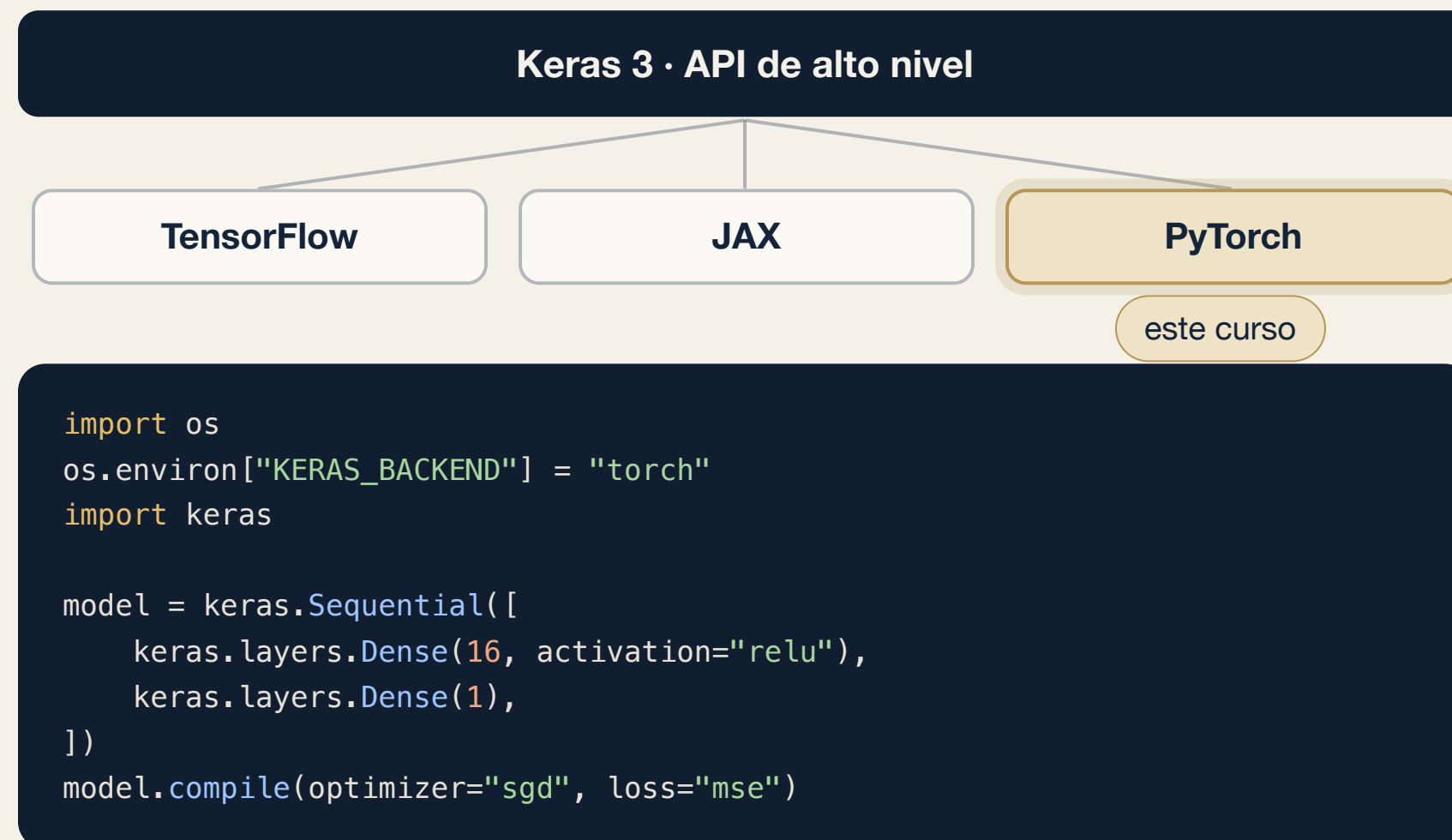
31

# Keras

Una API sencilla para armar y entrenar modelos.

Keras 3 corre sobre TensorFlow, JAX o PyTorch.

En el curso vamos a usar PyTorch directamente.



# Redes neuronales

---

¿Cómo ajustamos los parámetros de una red?

02

33

# Ingeniería de características

## FEATURE ENGINEERING



**Feature engineering:** nosotros diseñamos las características.

**Representation learning:** el modelo las aprende de los datos.

Una red profunda aprende varias transformaciones, una tras otra.

## REPRESENTATION LEARNING



la red aprende estas representaciones de los datos

34

# El perceptrón

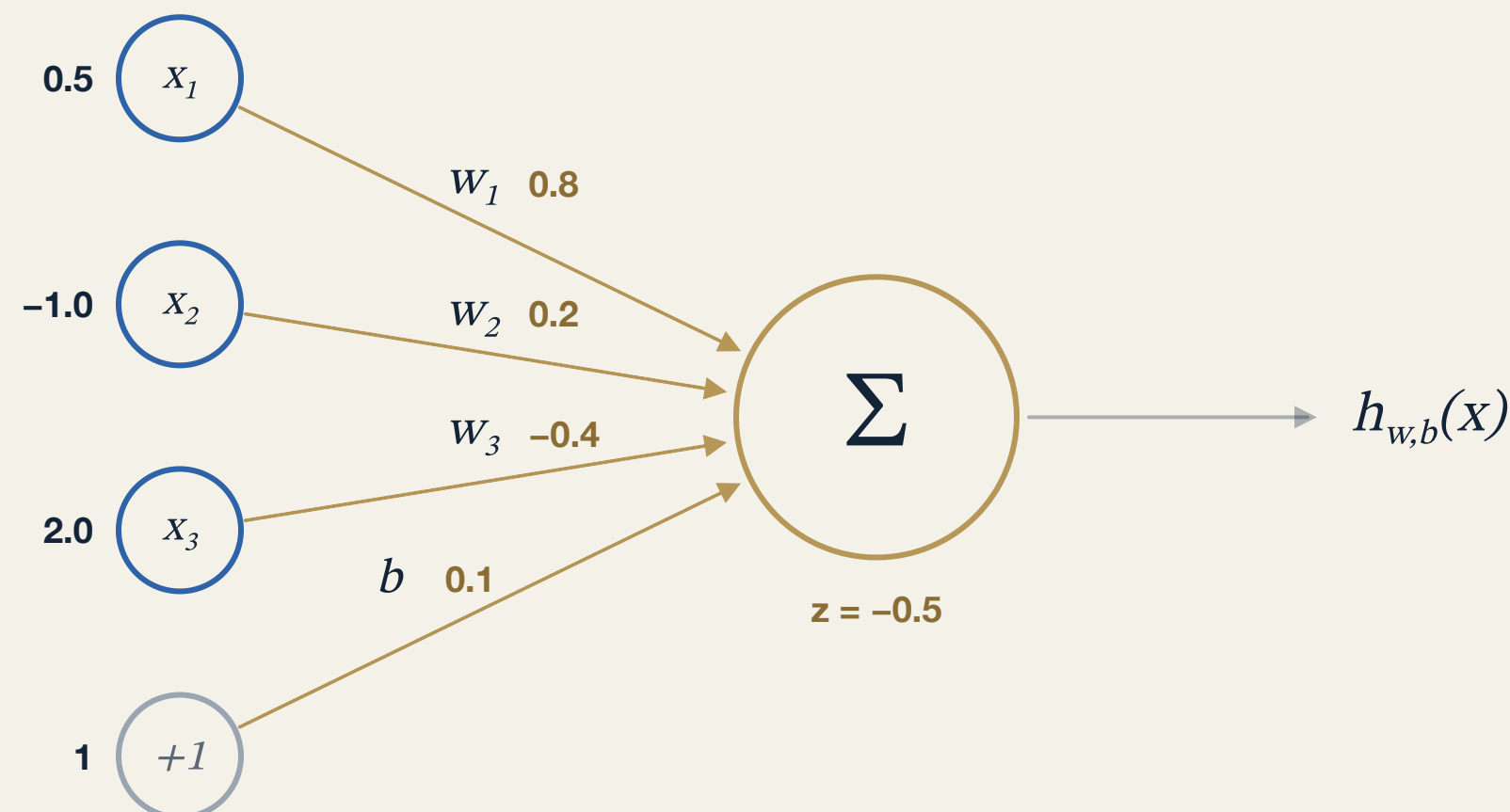
**Multiplica cada entrada por un peso, suma todo y agrega un sesgo.**

$$z = \sum_i w_i x_i + b$$

**EJEMPLO**

$$z = 0.5 \times 0.8 + (-1.0) \times 0.2 + 2.0 \times (-0.4) + 0.1$$

$$z = 0.4 - 0.2 - 0.8 + 0.1 = -0.5$$



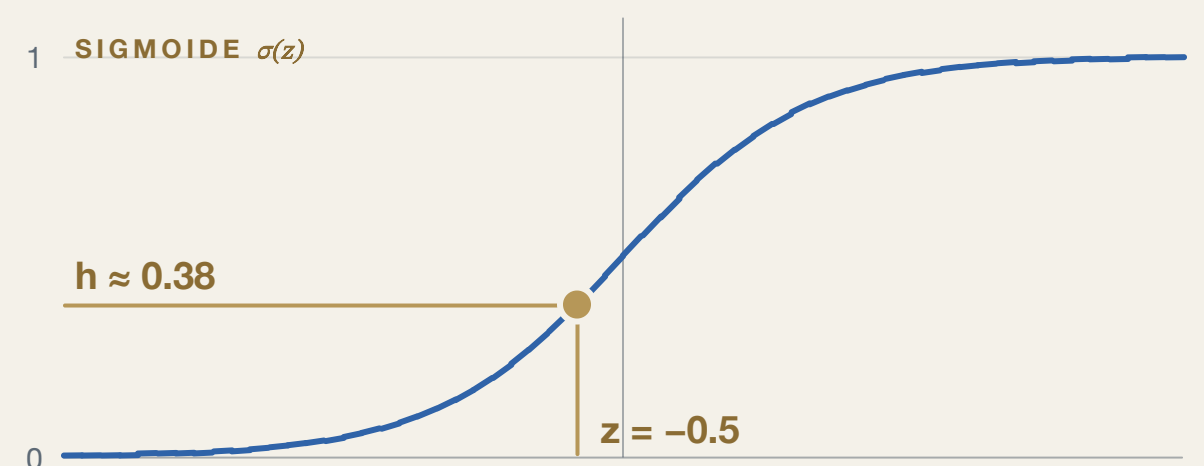
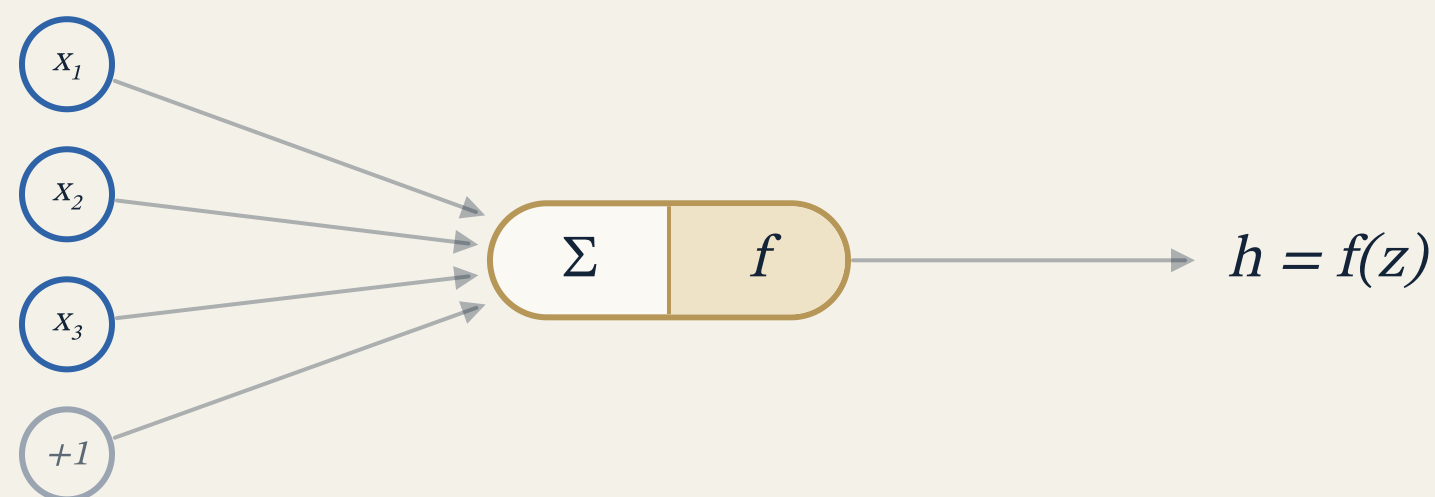
35

# La función de activación

A esa suma se le aplica la función de activación.

Si la función es no lineal, la red puede aprender relaciones no lineales.

$$h = f(z)$$



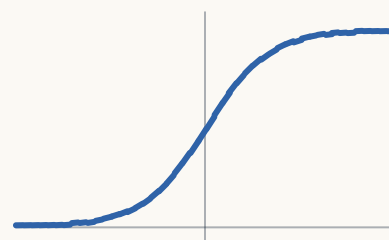
36

# Funciones de activación

## Sigmoide

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

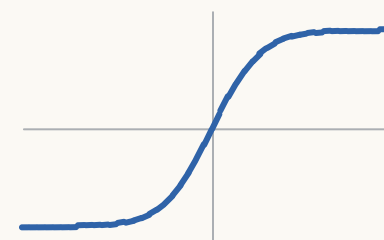
Salida en (0, 1)



## tanh

$$\tanh(x)$$

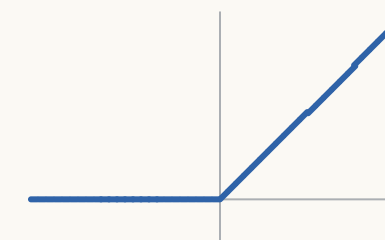
Salida en (-1, 1)



## ReLU

$$\max(0, x)$$

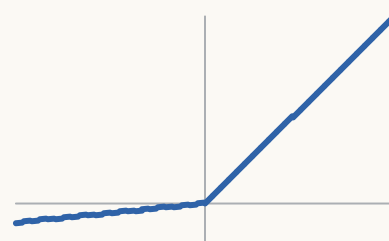
Salida en [0, ∞)



## Leaky ReLU

$$\max(0.1x, x)$$

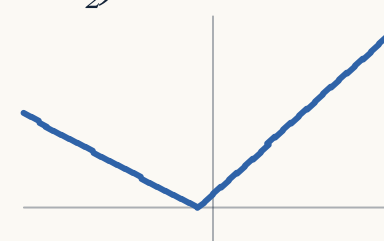
Pendiente 0.1 si  $x < 0$



## Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

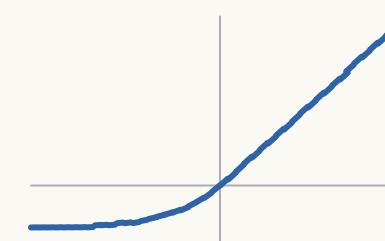
Aprende la forma (ejemplo)



## ELU

$$\begin{cases} x, & x \geq 0 \\ \alpha(e^x - 1), & x < 0 \end{cases}$$

Suave si  $x < 0 \cdot \alpha = 1$



La que escojan cambia cómo pasan la señal y los gradientes.

37

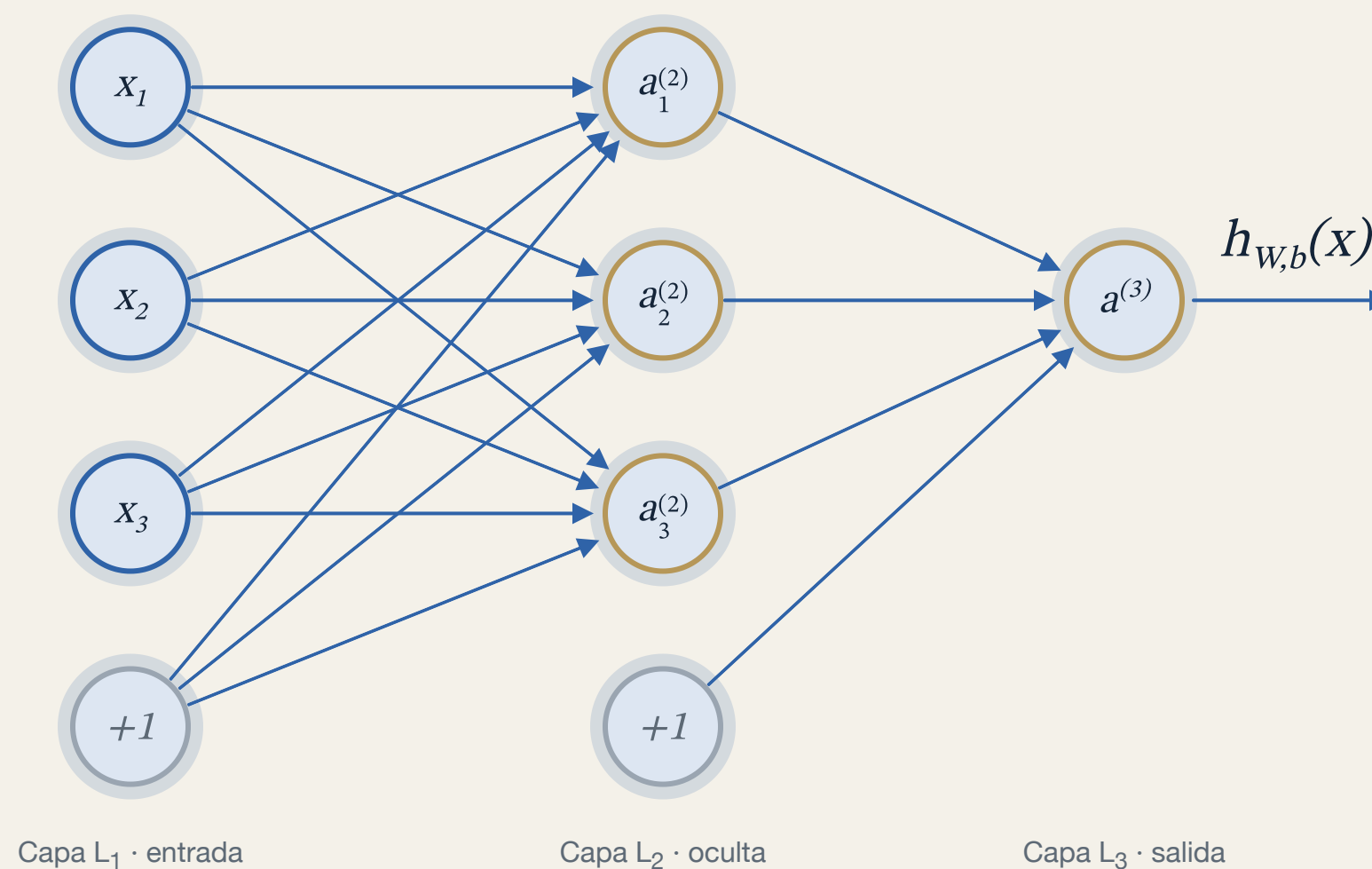
# Una red de neuronas

La salida de una capa es la entrada de la siguiente.

Cada capa cambia un poco más la representación.

$$a^{(2)} = f(W^{(1)}x + b^{(1)})$$

$$h_{w,b}(x) = f(W^{(2)}a^{(2)} + b^{(2)})$$



38

# Backpropagation

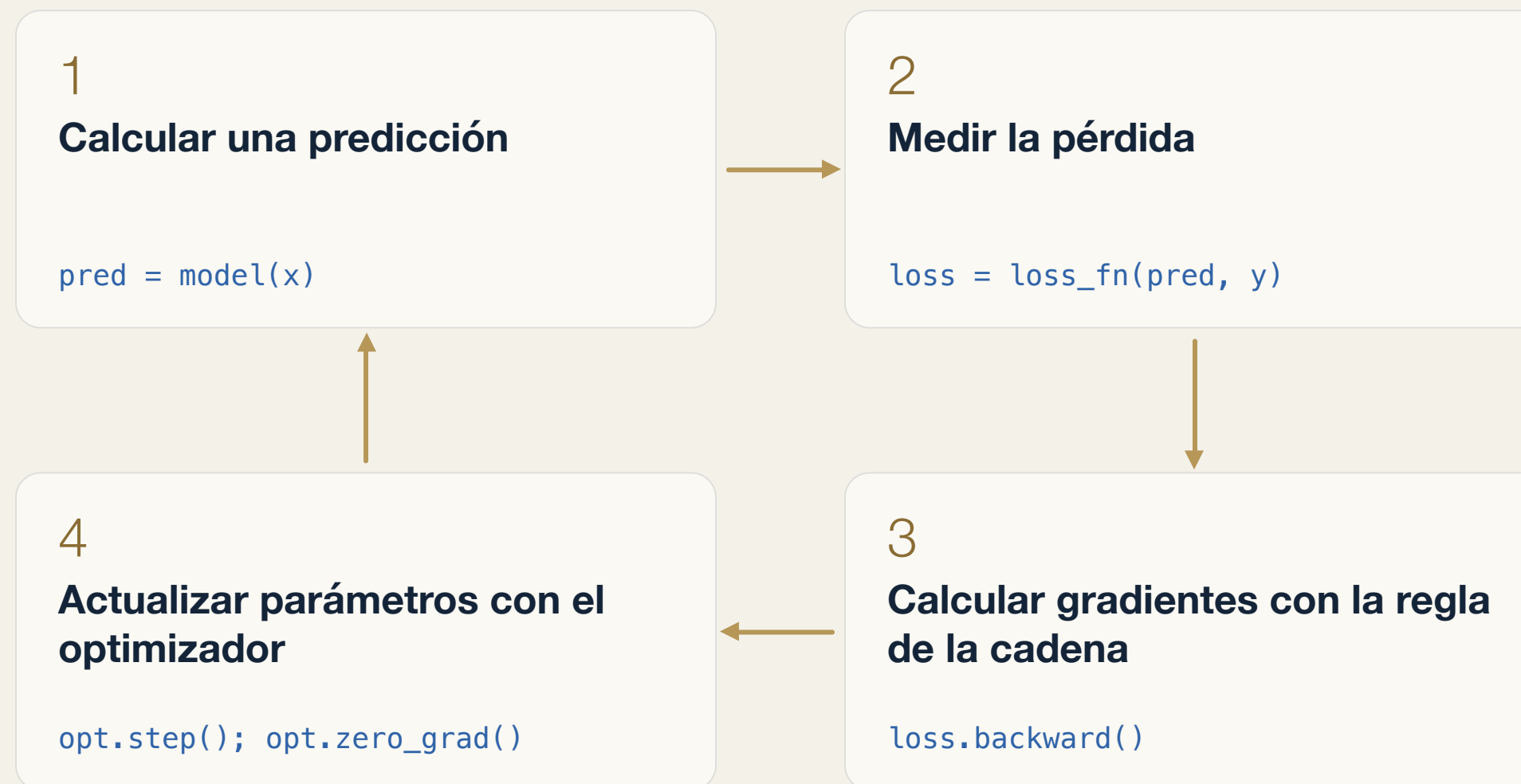
---

¿Cómo contribuye cada parámetro al error?

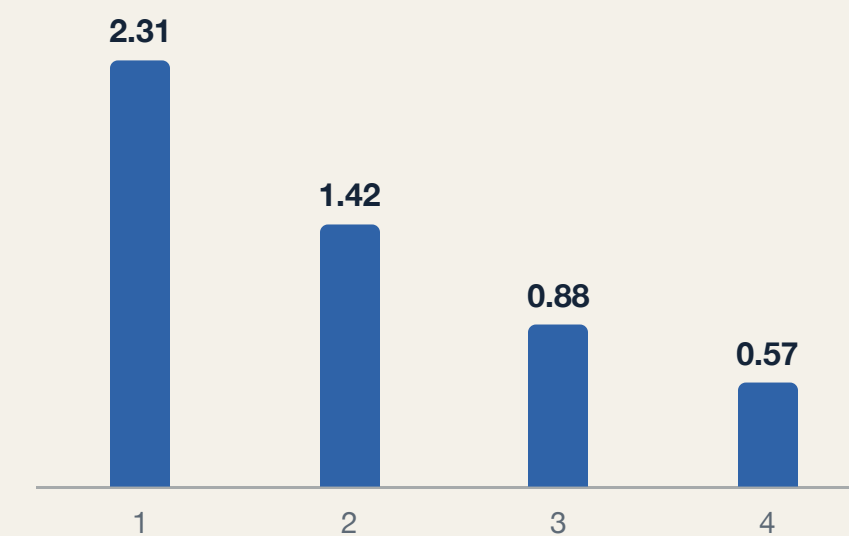
03

39

# El ciclo de entrenamiento



**PÉRDIDA POR ITERACIÓN (ILUSTRATIVA)**  
iteración 4



40

# El gradiente en cada capa

Backpropagation calcula la derivada de la pérdida respecto a cada peso.

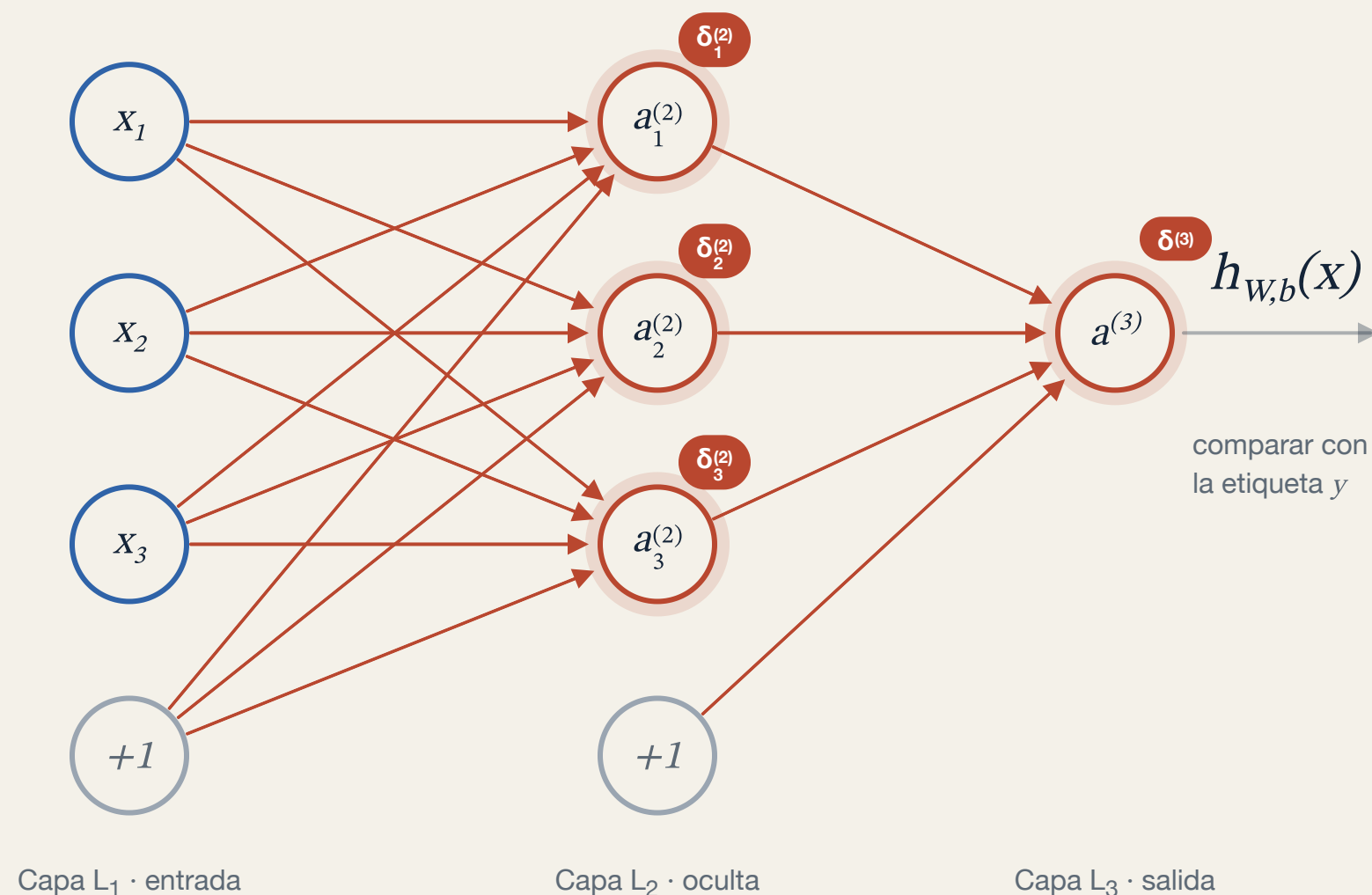
$\delta$  es el error que le toca a cada neurona.

Con esos gradientes, el optimizador actualiza los pesos.

$$\delta^{(2)} = ((W^{(2)})^T \delta^{(3)}) \odot f'(z^{(2)})$$

$$\partial J / \partial W^{(l)} = \delta^{(l+1)} (a^{(l)})^T$$

$$W \leftarrow W - \eta \partial J / \partial W$$



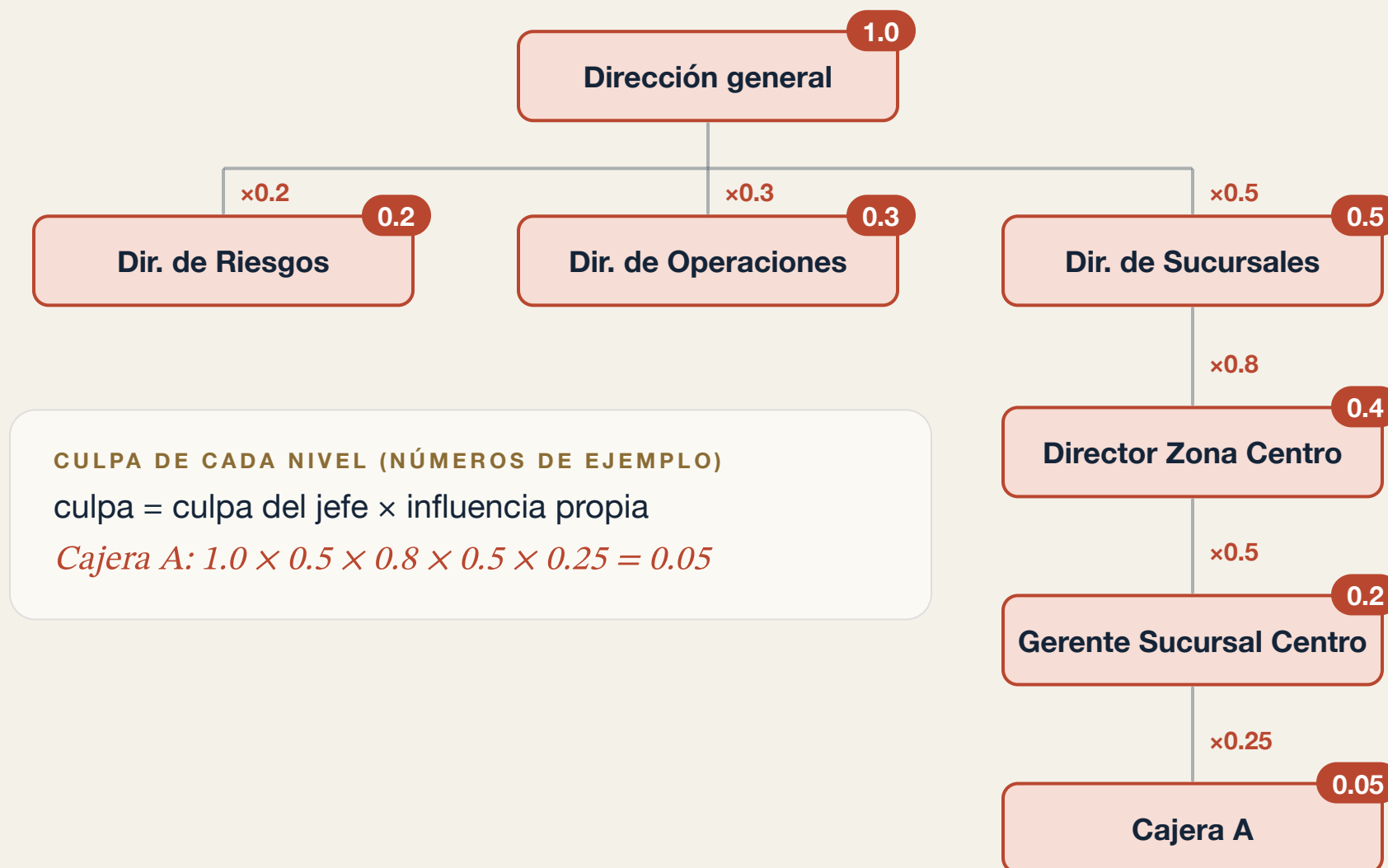
41

# Una analogía organizacional

Un mal resultado viene de decisiones en varios niveles.

La pregunta es cuánta culpa le toca a cada quien.

En una red, esa culpa se calcula con derivadas.



42

# Gradientes que se desvanecen

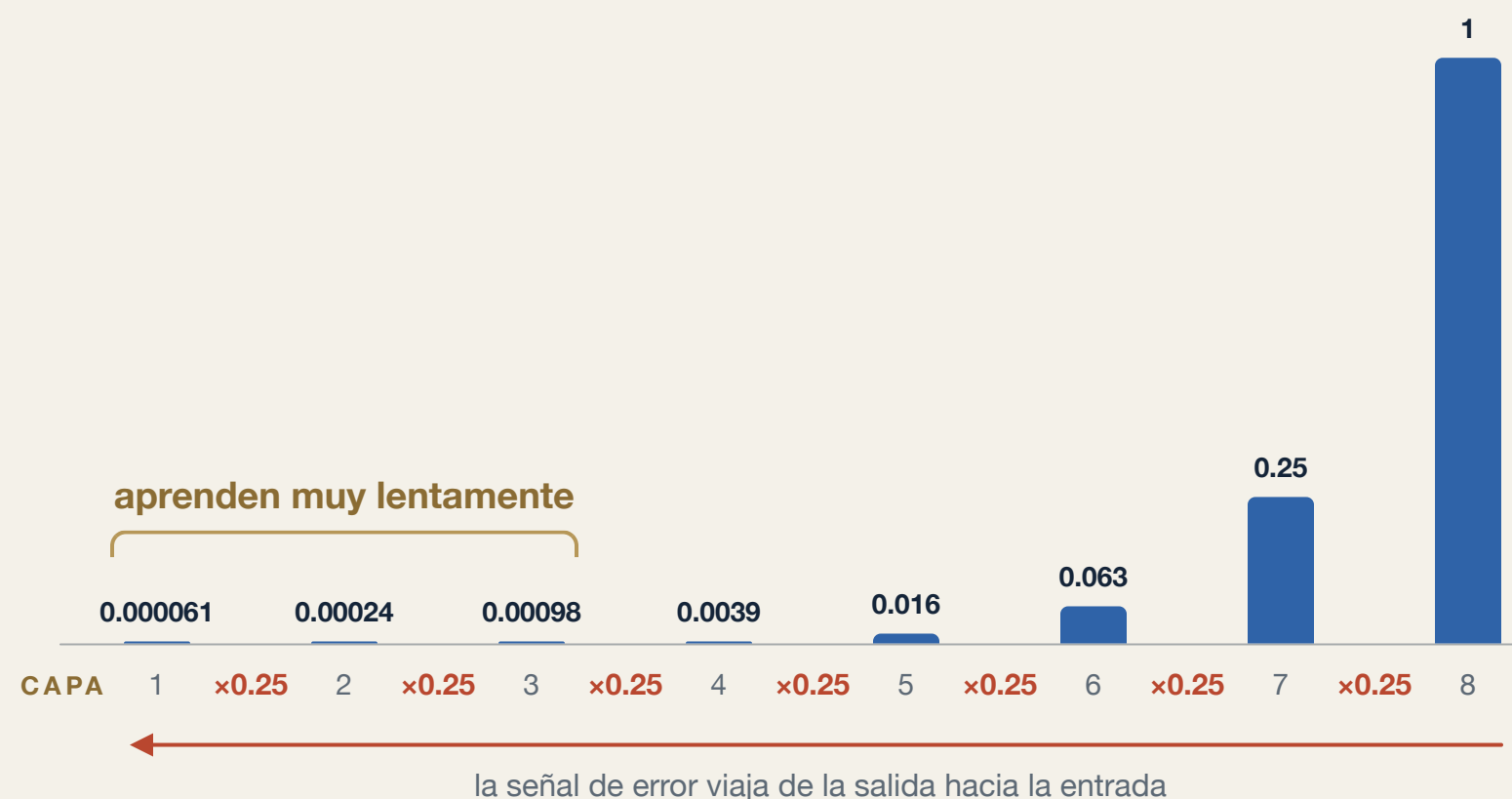
Si multiplicas muchas derivadas pequeñas, el gradiente se va a casi cero.

Las primeras capas casi no aprenden.

ReLU, una buena inicialización y las conexiones residuales ayudan.

Aun así, el problema no ha desaparecido del todo.

MAGNITUD DEL GRADIENTE POR CAPA · SIGMOIDE:  $f'(z) \leq 0.25$ , PESOS  $\approx 1$



ReLU y variantes

Inicialización cuidadosa

Conexiones residuales

43

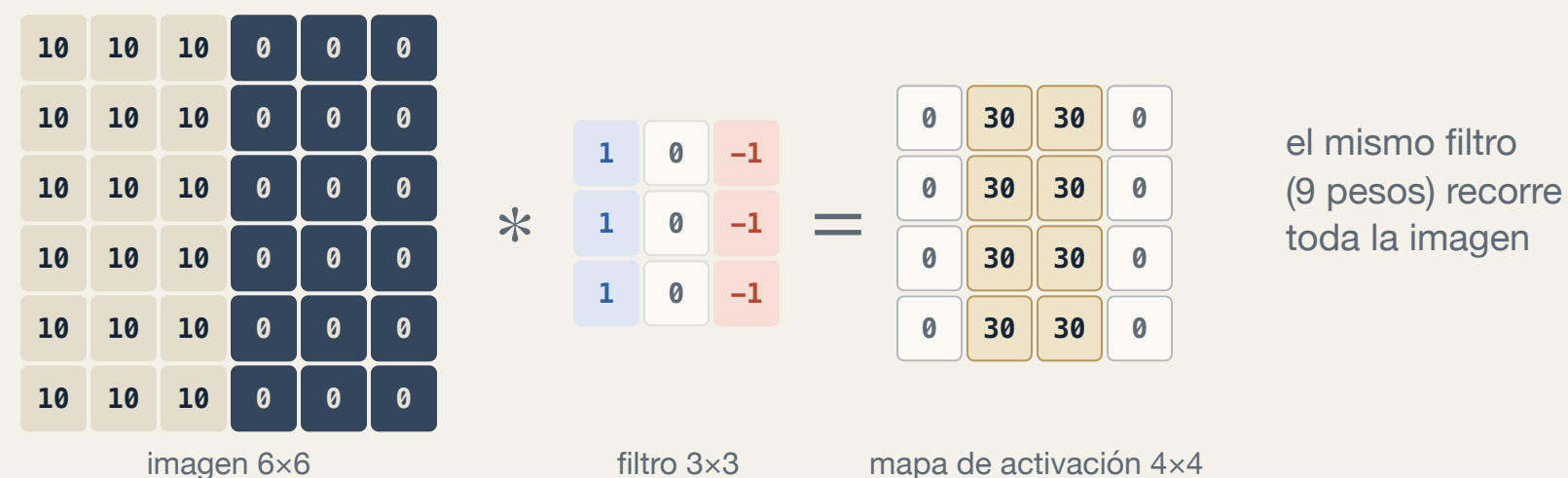
# Redes convolucionales

Usan filtros que buscan patrones pequeños en la imagen.

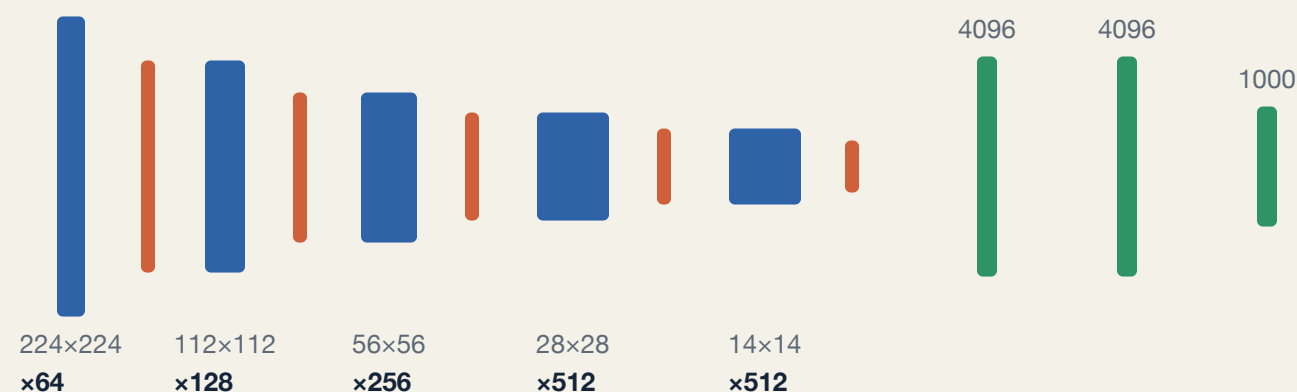
El mismo filtro recorre toda la imagen, así que hay pocos pesos.

Durante años fueron la arquitectura estándar en visión.

- convolución + ReLU
- max pooling
- totalmente conectada



**VGG-16: 13 CAPAS CONVOLUCIONALES + 3 TOTALMENTE CONECTADAS**



44

# Redes recurrentes

Leen una secuencia paso a paso y van guardando un estado.

Sirven para series de tiempo y cualquier dato que venga en orden.

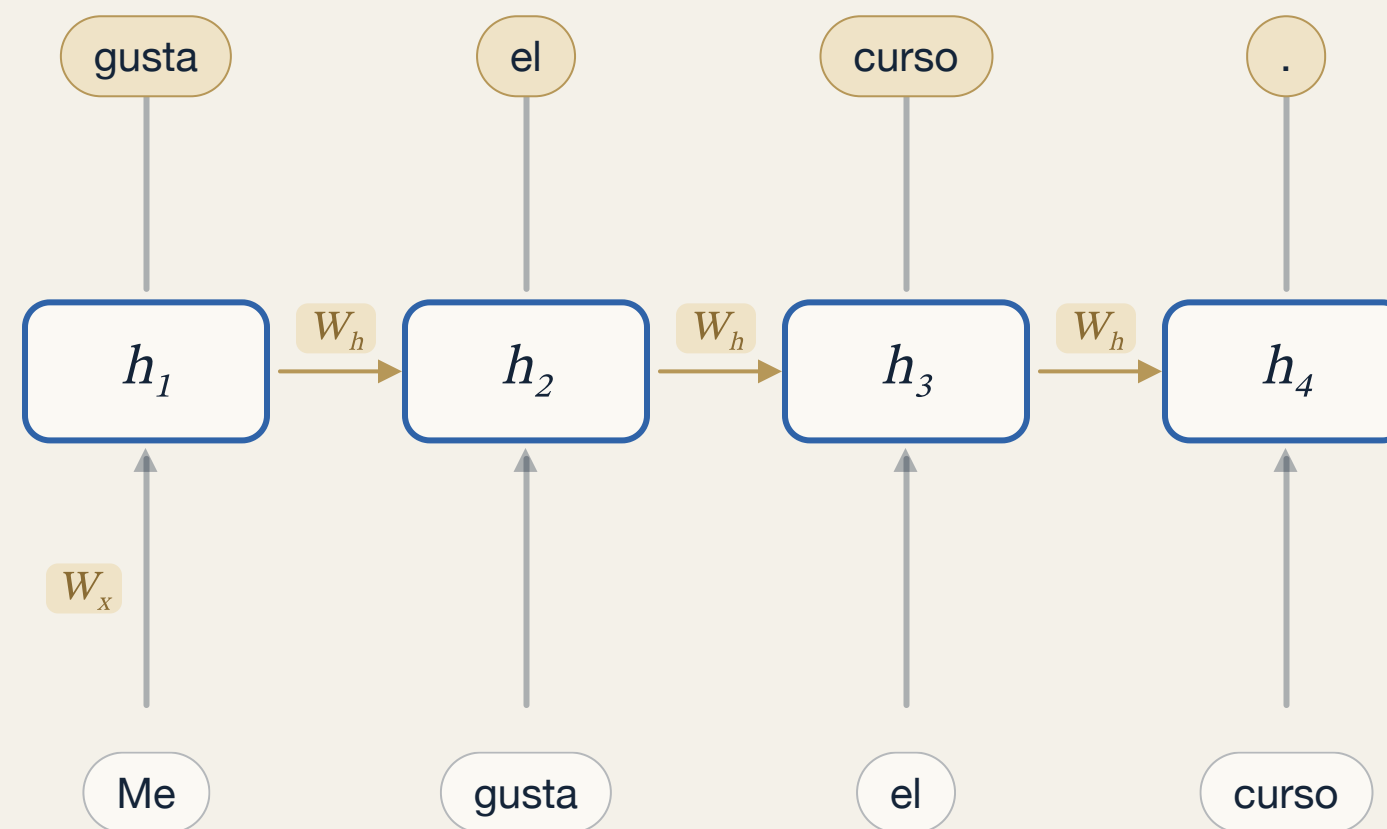
Para texto, hoy se usan más los Transformers.

series de tiempo

texto

audio

sensores



$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$$